



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 8

Finding analysis classes

A study material for the students of GLS University

UP Activity – Analyze a Use Case

The UP activity Analyze a Use Case produces two important outputs:

1. Analysis Classes

Classes that represent important concepts in the problem domain.

2. Use Case Realizations

Collaborations between objects that show how interacting objects realize the behavior expressed in a use case.

What is the Problem Domain?

The problem domain is the area in which the need for a software system arises.

Examples:

- Online selling
- Customer relationship management
- Banking
- Hotel reservation
- Airline reservation

The problem domain can also arise from physical hardware that requires software, as in an embedded system.

What is an Analysis Class?

An analysis class represents a crisp abstraction in the problem domain.

It should:

- Represent an important concept of the problem domain
- Map clearly to a real-world business concept
- Have a meaningful and appropriate name
- Represent one specific aspect of the problem domain

Examples

- Customer
- Product
- Account
- ShoppingBasket
- Order

These represent identifiable concepts within their respective problem domains.

Analysis Class and Problem Domain

Consider an E-Commerce System.

Problem Domain - Online Selling

Important Concepts -

- Customer
- Product
- ShoppingBasket
- Order

These concepts can become analysis classes because they represent identifiable elements of the problem domain.

Analysis Classes

Analysis focuses on the aspects of real-world things that are important from the perspective of the system.

For example, in a banking system, a customer may have many characteristics.

Relevant:

- Name
- Address
- Account information

Irrelevant to the banking system:

Preference for window or aisle seats on an aircraft

Therefore, only the information relevant to the system is modeled.

Analysis Class vs Design Class

Analysis Class -

Focuses on: What is important in the problem domain?

Contains:

- High-level attributes,
- High-level operations/responsibilities

Design Class -

Focuses on: How will the concept be implemented?

May contain:

- Detailed attributes
- Data types
- Visibility
- Method parameters
- Return types
- Implementation details

Problem Domain => Analysis Class => Design Class => Implementation

Anatomy of an Analysis Class

An analysis class contains a limited set of UML information.

Main elements -

- Name
- Attributes
- Operations

Additional elements may be included when useful:

- Visibility
- Stereotypes
- Tagged values

BankAccount – Analysis Class

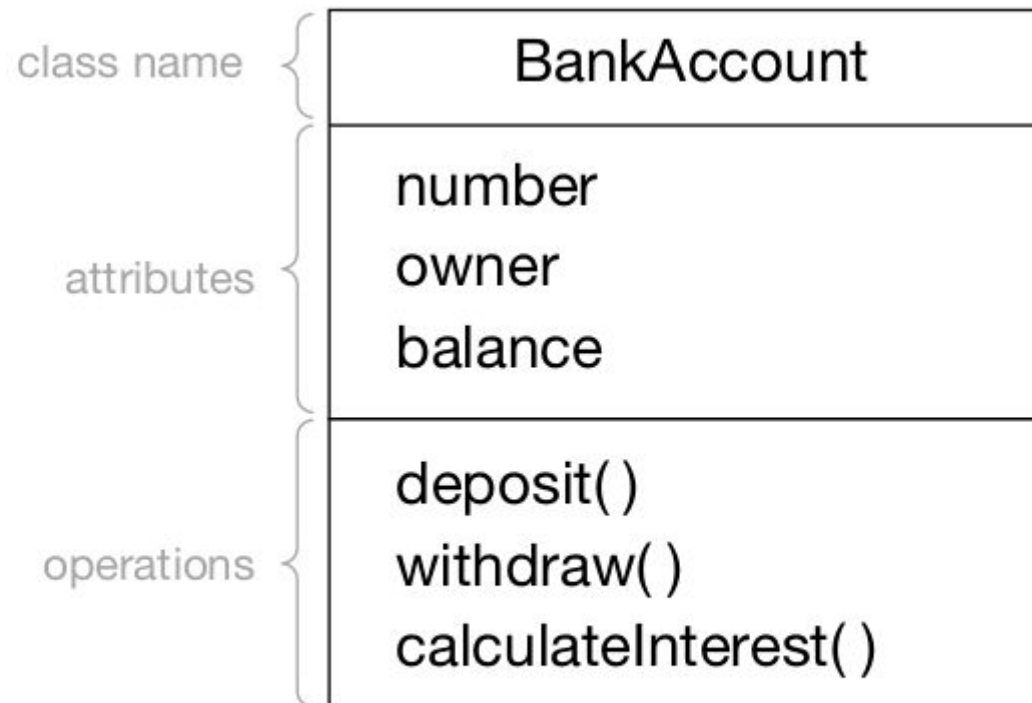


Figure 8.3

What Makes a Good Analysis Class?

A good analysis class should:

1. Have a clear, meaningful name.
2. Model one specific real-world concept.
3. Map to an identifiable business feature (e.g., Customer, ShoppingBasket).
4. Have a small, focused set of responsibilities (3–5).
5. Show high cohesion – all responsibilities serve one goal.
6. Have low coupling – not overly dependent on too many other classes.

Example:

ShoppingBasket → responsibilities: add item, remove item, show items.

(Cohesive → all about maintaining basket).

Wrong responsibilities for ShoppingBasket: validate credit card, print receipt (should go to other classes).

Rules of Thumb for Analysis Classes

Keep it simple:

- Each class should have 3 to 5 clear responsibilities (things it does).
- This keeps the class easy to understand and use.

Classes should work together:

- No class should exist alone.
- Every class should collaborate (work) with other classes to perform tasks.

Avoid too many small classes:

- If a class does only 1 or 2 small things, combine it with another related class.

Avoid very large classes:

- If a class has too many tasks, it becomes confusing.
- Split it into smaller, more focused classes.

Rules of Thumb for Analysis Classes

Avoid fake or meaningless classes:

- These are fake classes with only one method (like `dolt()` or `process()`), which just run code instead of modeling real-world things.
- They are not true objects.

Avoid “God classes”:

- These are classes that try to do everything (like one big “System” or “Controller” class).
- They make the design messy and hard to manage.
- Instead, divide the work among smaller, specific classes.

Keep inheritance simple:

- Don't make long chains of parent–child classes.
- Use inheritance only when there is a real “is-a” relationship (like Dog is a type of Animal).
- This keeps your class hierarchy meaningful and easy to follow.

Finding Classes

There is no single algorithm that always identifies the correct analysis classes.

Three important approaches are:

1. Noun/Verb Analysis

Analyze relevant text.

2. CRC Analysis

Use collaborative brainstorming.

3. Other Sources

Look at physical objects, paperwork, interfaces and conceptual entities.

Noun/Verb Analysis

A text-based technique to identify candidate classes, attributes, and responsibilities.

Rule of thumb:

- Nouns / noun phrases → possible classes or attributes.
- Verbs / verb phrases → possible responsibilities (operations).
- Works well because it uses the language of the problem domain.

Challenges:

- Synonyms (different words, same meaning) & homonyms (same word, different meaning).
- Hidden classes: important abstractions that are not explicitly mentioned.
- Example: In a holiday reservation system, “Order” may be missing but is crucial.

Noun/Verb Analysis

Procedure -

Collect relevant information



Identify nouns and noun phrases



Identify verbs and verb phrases



Clarify unknown terms



Resolve synonyms and homonyms



Create candidate classes, attributes and responsibilities



Allocate attributes and responsibilities to classes



Identify candidate relationships



Create first-cut class model

CRC Analysis (Class, Responsibilities, Collaborators)

CRC is a brainstorming technique for identifying important concepts in the problem domain.

CRC uses sticky notes containing three sections:

Class name (top)

Responsibilities (left side)

Collaborators (right side → other classes it works with)

Example -

BankAccount

Responsibility: Maintain balance

Collaborator: Bank

CRC

CRC Procedure

Phase 1: Brainstorm (Gather Info)

Participants: OO analysts + stakeholders + domain experts.

Rules:

All ideas are accepted (no arguments, just record everything).

Steps:

Ask team to name business “things” (e.g., Customer, Product). → Write on sticky notes.

Note down responsibilities on the sticky notes.

Identify collaborating classes (draw connections or note them).

CRC

Phase 2: Analyze (Refine Info)

Participants: OO analysts + domain experts.

Decide which notes should be:

Classes (key business concepts, independent entities).

Attributes (parts of another class).

Rule: If not sure → treat it as a class (can refine later).

Other Sources of Classes

Beyond text analysis, look at:

- Physical objects – e.g., Aircraft, Hotel, Person.
- Paperwork – e.g., Invoice, Order, Bankbook.
- Be careful → don't just copy outdated or unnecessary processes.
- Interfaces – e.g., Screens, Keyboards, External systems.
- Conceptual entities – important but abstract ideas.
- Example: LoyaltyProgram (not physical, but essential to model).

THANK YOU